

# Web and Data Engineering

---

## Vorlesung 06: Data Engineering

Prof. Dr. Michael Granitzer

25 März 2026

# Data Engineering

Die Vorlesung verknüpft datengetriebene Web-Systeme mit Pipelines, Speicherarchitekturen und Betriebsfragen rund um Qualität und Skalierung.

## Fokus

- wie aus Rohdaten verarbeitbare Datenprodukte werden
- welche Speicher- und Verarbeitungsmodelle Data Engineering prägen
- warum Batch- und Stream-Verarbeitung unterschiedliche Zuschnitte brauchen
- wie Datenqualität, Governance und Betrieb zusammenwirken

# Modul 1

---

## Datenpipelines und Datenprodukte

# Leitfrage

Was ist Big Data und warum überfordert es klassische Systeme?

# Welche Daten sammeln wir überhaupt?

Moderne Unternehmen und Plattformen erzeugen Daten aus sehr unterschiedlichen Quellen:

## Operative Systeme

- **Case Management:** Audit-Trails, Compliance-Daten
- **CRM / ERP:** SAP, Oracle — Kundendaten, Transaktionen
- **Service Desk:** Tickets, Vorfälle, Reaktionszeiten

## Digitale & physische Welt

- **Internet Clickstreams:** Seitenaufrufe, Klicks, Sessions
- **Netflow:** Netzwerkverkehr, Sicherheitsereignisse
- **IoT Device Events:** Sensoren, Maschinen, Fahrzeuge

Jede dieser Quellen hat eigenes Format, eigene Geschwindigkeit, eigene Vertrauenswürdigkeit.

# Was macht Daten zu „Big Data“?

## Die Fünf Vs von Big Data (IBM)

Big Data ist kein Größenbegriff, sondern ein Eigenschaftsprofil: **Volume, Variety, Velocity, Veracity, Value.**

Die klassische Datenbank-Annahme: Daten passen in Tabellen, kommen in moderatem Tempo, und sind im Wesentlichen korrekt. Alle fünf Vs stellen genau diese Annahmen in Frage.

# V1 — Volume: Das Mengenproblem

Datenmengen übersteigen, was einzelne Maschinen verarbeiten können.

## Quellen für extreme Volumina

| Bereich          | Beispiel                                  |
|------------------|-------------------------------------------|
| Web & E-Commerce | Produktaufrufe, Kaufhistorien, Reviews    |
| Banking          | Zahlungstransaktionen weltweit            |
| Social Networks  | Posts, Likes, Kommentare                  |
| LTE / 5G         | Verbindungsprotokolle, Netzwerktelemetrie |

## Einordnung

*„Eine einzelne Google-Suche nutzt mehr Rechenleistung als die gesamte Apollo-Raumfahrtmission.“*

- Google indiziert ~10 Mrd. Seiten = ca. 200 TB
- Bei 50 MB/s Bandbreite: Lesen allein würde **40 Tage** dauern
- Lösung: Parallelisierung und verteilte Verarbeitung

RDBMS wurden für einzelne, skalierbare Maschinen entworfen. Horizontale Verteilung ist nachträglich schwer einzubauen.

# V2 — Variety: Das Formatproblem

Daten kommen nicht mehr nur als Tabellen.

| Datenmodell       | Beschreibung                                   | Beispiel                     |
|-------------------|------------------------------------------------|------------------------------|
| Relational        | Tabellen, Transaktionen, festes Schema         | Bestellungen, Konten         |
| Text              | Freitextinhalt, unstrukturiert                 | Web-Seiten, E-Mails, Tickets |
| Semi-strukturiert | Schema flexibel, hierarchisch                  | XML, JSON, Logs              |
| Graph             | Knoten und Kanten, Beziehungsnetze             | Soziale Netzwerke, RDF       |
| Streaming         | Kontinuierlicher Datenstrom, nur einmal lesbar | Sensor-Events, Klicks        |

Wenn Quellsysteme ihr Format ändern, bricht nachgelagerte Verarbeitung. **Schema-Evolution** ist ein zentrales Problem im Data Engineering.

# V3 — Velocity: Das Geschwindigkeitsproblem

Daten erscheinen mit sehr unterschiedlichen Geschwindigkeiten — und Entscheidungen müssen entsprechend zeitnah getroffen werden.

## Konkrete Zahlen

| Quelle                    | Geschwindigkeit           |
|---------------------------|---------------------------|
| Globaler Internettraffic  | ~50.000 GB/s              |
| Twitter                   | ~6.000 Tweets/Sekunde     |
| Suchanfragen (Google)     | ~100.000/Sekunde          |
| Kreditkartentransaktionen | tausende/Sekunde weltweit |

## Konsequenz für Systeme

- Systeme müssen Entscheidungen in **Millisekunden** bis Sekunden treffen
- Betrugserkennung kann nicht auf den nächsten Batch-Job warten
- Alerting auf Netzwerkanomalien erfordert Echtzeit-Verarbeitung

# V4 — Veracity: Das Vertrauensproblem

Nicht alle Daten sind verlässlich. Veracity beschreibt den Grad der Unsicherheit.

## Quellen von Datenunsicherheit

- Fehlerhafte Eingaben (Tippfehler, falsche Werte)
- Veraltete oder doppelte Datensätze
- Inkonsistente Definitionen zwischen Systemen
- Messfehler bei Sensoren
- Absichtlich verfälschte Daten (Fraud)

## Empirische Einordnung

Laut Unternehmensumfragen zweifelt **jeder dritte Entscheidungsträger** an der Qualität der Daten, auf denen Entscheidungen basieren.

Unsichere Daten führen zu unsicheren Modellen — unabhängig von Rechenleistung oder Algorithmus.

**Garbage In, Garbage Out:** Kein ML-Modell und kein Analysesystem kann systematisch fehlerhafte Eingaben kompensieren.

# V5 — Value: Das Nutzenproblem

Daten allein schaffen keinen Wert. Großmengen zu speichern ist kostspielig — der Nutzen muss aktiv erschlossen werden.

## Anwendungsfelder mit nachgewiesenem Wert

| Domäne              | Anwendung                              |
|---------------------|----------------------------------------|
| E-Commerce / Retail | Dynamic Pricing, Customer Segmentation |
| Industrie           | Predictive Maintenance                 |
| Landwirtschaft      | Smart Farming (Ertragsprognosen)       |
| Banken              | Kreditrisikomodelle, Betrugserkennung  |
| Wissenschaft        | Genomik, Klimamodellierung             |

## Wie Wert entsteht

- **Data Science:** Muster erkennen, Prognosen ableiten
- **Data Mining:** Strukturen in großen Datensätzen finden
- **Machine Learning:** Modelle auf historischen Daten trainieren

Wert entsteht durch die  
**Kombination**  
aus Daten, Methoden und fachlichem Kontext.

# Warum klassische Datenbanken scheitern

Klassische RDBMS wurden für eine Welt entworfen, die es so nicht mehr gibt:

| Annahme klassischer Systeme        | Realität bei Big Data                      |
|------------------------------------|--------------------------------------------|
| Daten passen auf eine Maschine     | Petabyte-Mengen erfordern Cluster          |
| Schema ist stabil und bekannt      | Format ändert sich laufend (Variety)       |
| Schreibvorgänge sind moderat       | Millionen Events pro Sekunde (Velocity)    |
| Daten sind im Wesentlichen korrekt | Systematische Qualitätsprobleme (Veracity) |
| Ausfälle sind Ausnahmen            | Bei 1000 Knoten: täglich Ausfälle          |

Die Fünf Vs erzwingen einen Paradigmenwechsel: von zentralisierten RDBMS zu verteilten, skalierbaren Systemen mit expliziter Qualitätskontrolle.

# Mini-Übung: Die 5 Vs im E-Commerce

Ordnen Sie jedem V ein konkretes Beispiel aus einem Online-Shop zu.

| V        | Ihre Zuordnung |
|----------|----------------|
| Volume   | ?              |
| Variety  | ?              |
| Velocity | ?              |
| Veracity | ?              |
| Value    | ?              |

# Musterlösung

| V               | E-Commerce-Beispiel                                                                     |
|-----------------|-----------------------------------------------------------------------------------------|
| <b>Volume</b>   | Klick-Logs aller Seitenbesuche (Milliarden Einträge/Tag)                                |
| <b>Variety</b>  | Produktdaten (strukturiert), Bewertungstexte (Text), Nutzersessions (Semi-strukturiert) |
| <b>Velocity</b> | Preisanpassungen in Echtzeit bei Flash-Sales                                            |
| <b>Veracity</b> | Fake-Bewertungen, fehlerhafte Produktattribute, doppelte Kundendatensätze               |
| <b>Value</b>    | Empfehlungsalgorithmus steigert durchschnittlichen Warenkorbwert                        |

Big Data ist kein Mengenproblem allein. Die fünf Vs beschreiben ein Eigenschaftsprofil, das klassische Datenbanken systematisch überfordert. Jedes V erfordert andere architektonische Antworten.

# Schlüsselbegriffe — Modul 1

| Begriff            | Erklärung                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------|
| <b>Big Data</b>    | Datensätze, die durch Volume, Variety, Velocity, Veracity oder Value klassische Verarbeitung überfordern |
| <b>Volume</b>      | Datenmenge — von Gigabyte bis Petabyte und darüber hinaus                                                |
| <b>Variety</b>     | Vielfalt der Datenformate und -modelle (relational, Text, Graph, Stream)                                 |
| <b>Velocity</b>    | Geschwindigkeit, mit der Daten erzeugt und verarbeitet werden müssen                                     |
| <b>Veracity</b>    | Verlässlichkeit und Qualität der Daten; Unsicherheit über Korrektheit                                    |
| <b>Value</b>       | Nutzen, der durch Analyse und Anwendung aus Daten gewonnen wird                                          |
| <b>ETL</b>         | Extract, Transform, Load — klassischer Drei-Schritt-Prozess der Datenpipeline                            |
| <b>Clickstream</b> | Protokoll der Aktionen eines Nutzers auf einer Website                                                   |

# Modul 2

---

## Speicher- und Verarbeitungsschichten

# Leitfrage

Wie gewinnt man Wissen aus Daten?

# Data Science — eine Einordnung

**Data Science** erforscht und wendet Technologien, Systeme und Algorithmen an, um Wissen aus potenziell großen, heterogenen und dynamisch veränderlichen Datensätzen zu gewinnen.

## Data Science

Methoden und Systeme zur Wissensgewinnung aus Daten — verbindet Statistik, Informatik und Domänenwissen.

## Machine Learning

Algorithmen, die aus vergangener Erfahrung lernen und Entscheidungen treffen, ohne explizit programmiert zu sein.

## Artificial Intelligence

Systeme, die intelligentes Verhalten zeigen — ML ist der aktuell dominante Ansatz, AI der übergeordnete Begriff.

# Deduktives vs. Induktives Schließen

Die Art, wie Wissen gewonnen wird, unterscheidet sich grundlegend:

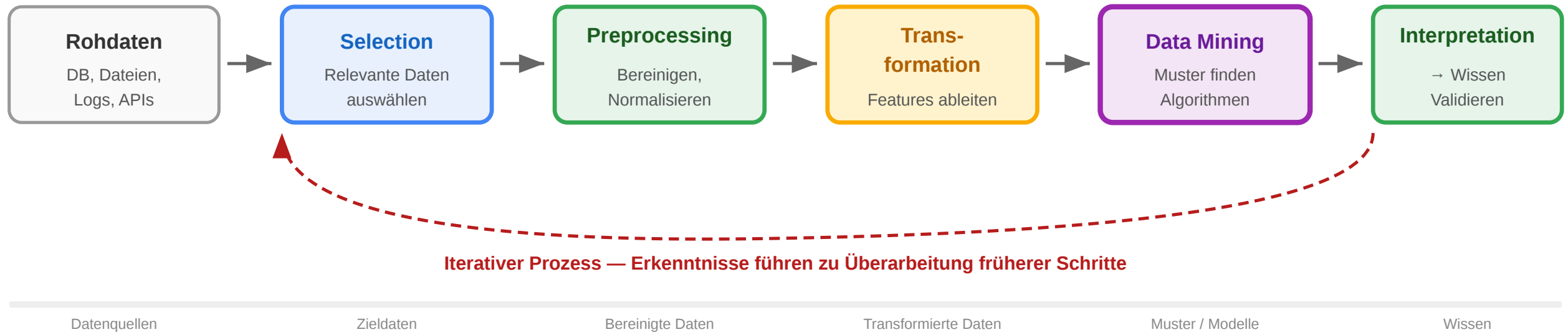
| Aspekt        | Deduktives Schließen                                                               | Induktives Schließen                                                       |
|---------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Ausgangspunkt | Gegebene Prämissen und Regeln                                                      | Beobachtete Datenmuster                                                    |
| Richtung      | Allgemein → Spezifisch                                                             | Spezifisch → Allgemein                                                     |
| Ergebnis      | Logisch notwendige Schlussfolgerungen                                              | Wahrscheinliche Verallgemeinerungen                                        |
| Beispiel      | „Alle Menschen sind sterblich. Sokrates ist ein Mensch. → Sokrates ist sterblich.“ | „Diese 10.000 Kunden kauften nach A auch B. → Empfehle B nach Kauf von A.“ |
| Einsatz in DS | Regelbasierte Systeme, Constraint-Solving                                          | Machine Learning, Data Mining                                              |

**Data Science ist primär induktiv:** Muster werden aus Daten extrahiert — keine vollständige Gewissheit, aber probabilistisch fundierte Aussagen.

# Der KDD-Prozess

Knowledge Discovery in Databases (KDD) beschreibt den vollständigen Weg von Rohdaten zum Wissen:

## KDD-Prozess (Knowledge Discovery in Databases)



- **Selection:** Auswahl relevanter Daten aus dem Gesamtbestand
- **Preprocessing:** Fehlende Werte, Duplikate, Ausreißer behandeln
- **Transformation:** Merkmale ableiten, Dimensionsreduktion
- **Data Mining:** Algorithmen auf transformierten Daten anwenden
- **Interpretation:** Gefundene Muster fachlich bewerten und validieren

In der Praxis wird der Prozess mehrfach durchlaufen. Neue Erkenntnisse im Mining-Schritt führen häufig zur Überarbeitung der Preprocessing-Schritte.



# Data Mining Aufgaben im Überblick

| Kategorie   | Sub-Aufgabe             | Beschreibung                                | Beispiel                   |
|-------------|-------------------------|---------------------------------------------|----------------------------|
| Predictive  | Classification          | Klasse aus bekannten Attributen vorhersagen | Spam-Filter, Kreditrisiko  |
| Predictive  | Regression              | Numerischen Wert vorhersagen                | Preisvorhersage, Nachfrage |
| Predictive  | Collaborative Filtering | Präferenzen aus ähnlichen Nutzern ableiten  | Film-/Produktempfehlungen  |
| Descriptive | Clustering              | Ähnliche Objekte ohne Klassen gruppieren    | Kundensegmentierung        |
| Descriptive | Association Rules       | Häufig gemeinsam auftretende Muster         | Marktkorb-Analyse          |
| Descriptive | Sequential Patterns     | Zeitlich geordnete Muster                   | Klickpfad-Analyse          |
| Explorative | Anomaly Detection       | Abweichungen von Normalverhalten finden     | Betrugserkennung           |

# Classification: Vorhersage mit bekannten Klassen

## Prinzip

1. **Trainingsdaten:** Datensatz mit bekanntem Klassen-Attribut
2. **Modelltraining:** Algorithmus lernt Entscheidungsregeln
3. **Testdaten:** Modell auf unbekannte Daten anwenden
4. **Evaluation:** Genauigkeit messen (Accuracy, F1, etc.)

## Beispiel: Entscheidungsbaum

```
Alter > 30?  
├── Ja → Einkommen > 40k?  
│   ├── Ja → Kauf wahrscheinlich ✓  
│   └── Nein → Kauf unwahrscheinlich ✗  
└── Nein → Kauf unwahrscheinlich ✗
```

## Einsatz

- E-Mail: Spam oder kein Spam?
- Medizin: Diagnose aus Symptomen
- Marketing: Kauft dieser Kunde?
- Kredit: Kreditausfall wahrscheinlich?

## Algorithmen

- Decision Trees
- Random Forests
- Support Vector Machines
- Neural Networks

# Clustering: Gruppierung ohne Klassen

Clustering findet natürliche Gruppen in Daten — ohne vorgegebene Klassen.

## Beispiel: Kundensegmentierung

Cluster A: Jung, hohes Einkommen, Hochschule  
→ Premium-Angebote, digitale Kanäle

Cluster B: Mittleres Alter, mittleres Einkommen  
→ Familienprodukte, Newsletter

Cluster C: Senioren, niedriges Einkommen  
→ Basistarife, persönliche Beratung

## Eigenschaften

- kein Klassen-Attribut notwendig
- Anzahl der Cluster oft vorab wählen (k-Means)
- Ergebnisse erfordern fachliche Interpretation

## Algorithmen

- k-Means (Partitionierung)
- DBSCAN (Dichtebasiert)
- Hierarchisches Clustering

# Association Rule Mining

Findet Produkte oder Ereignisse, die häufig gemeinsam auftreten.

## Marktkorb-Analyse

| Transaktion | Produkte       |
|-------------|----------------|
| T1          | p1, p3, p5, p8 |
| T2          | p2, p5, p8     |
| T3          | p1, p5, p8     |
| T4          | p3, p4         |

Regel:

$\{p5\} \rightarrow \{p8\}$

mit Support 75%, Confidence 100%

→ Wer p5 kauft, kauft fast immer auch p8.

## Anwendungen

- **Retail:** Platzierung verwandter Produkte
- **Online-Shop:** „Kunden, die X kauften, kauften auch Y“
- **Medizin:** Symptom-Diagnose-Korrelationen

## Kennzahlen

- **Support:** Wie oft tritt die Regel auf?
- **Confidence:** Wie zuverlässig ist die Regel?
- **Lift:** Wie viel besser als Zufall?

# Text Mining und Graph Mining

## Text Mining

Klassische Data-Mining-Methoden auf Textdaten:

- **Web-Seiten clustern:** Ähnliche Themen automatisch gruppieren
- **Seiten klassifizieren:** Spam, Kategorie, Sentiment
- **Named Entity Recognition:** Personen, Orte, Unternehmen
- **Topic Modeling:** Latente Themenstrukturen finden

Herausforderung: Texte sind hochdimensional und dünn besetzt.

## Graph Mining

Muster in Netzwerkdaten erkennen:

- **Community Detection:** Gruppen in sozialen Netzwerken
- **Link Prediction:** Wer wird mit wem verbunden?
- **Anomalie-Erkennung:** Ungewöhnliche Verbindungsmuster
- **Pagerank:** Zentrale Knoten im Netzwerk

Beispiel: Betrugserkennung über ungewöhnliche Transaktionsnetzwerke.

# Mini-Übung: Mining-Aufgaben klassifizieren

Ordnen Sie die folgenden Anwendungen einer Mining-Kategorie zu: **Classification**, **Clustering**, **Regression** oder **Collaborative Filtering**.

| Anwendung                                                       | Kategorie |
|-----------------------------------------------------------------|-----------|
| Netflix empfiehlt Serien basierend auf ähnlichen Nutzerprofilen | ?         |
| Ein Modell schätzt den Verkaufspreis eines Hauses               | ?         |
| Kunden werden nach Kaufverhalten in Gruppen eingeteilt          | ?         |
| Eine Bank entscheidet: Kreditantrag genehmigen oder ablehnen?   | ?         |

# Musterlösung

| Anwendung                                                       | Kategorie                      |
|-----------------------------------------------------------------|--------------------------------|
| Netflix empfiehlt Serien basierend auf ähnlichen Nutzerprofilen | <b>Collaborative Filtering</b> |
| Ein Modell schätzt den Verkaufspreis eines Hauses               | <b>Regression</b>              |
| Kunden werden nach Kaufverhalten in Gruppen eingeteilt          | <b>Clustering</b>              |
| Eine Bank entscheidet: Kreditantrag genehmigen oder ablehnen?   | <b>Classification</b>          |

Data Science ist die Disziplin, die aus Daten belastbares Wissen macht — durch den KDD-Prozess, induktive Lernmethoden und iterative Modellentwicklung. Wer die Aufgabe falsch wählt, erhält richtige Antworten auf die falsche Frage.

# Modul 3

---

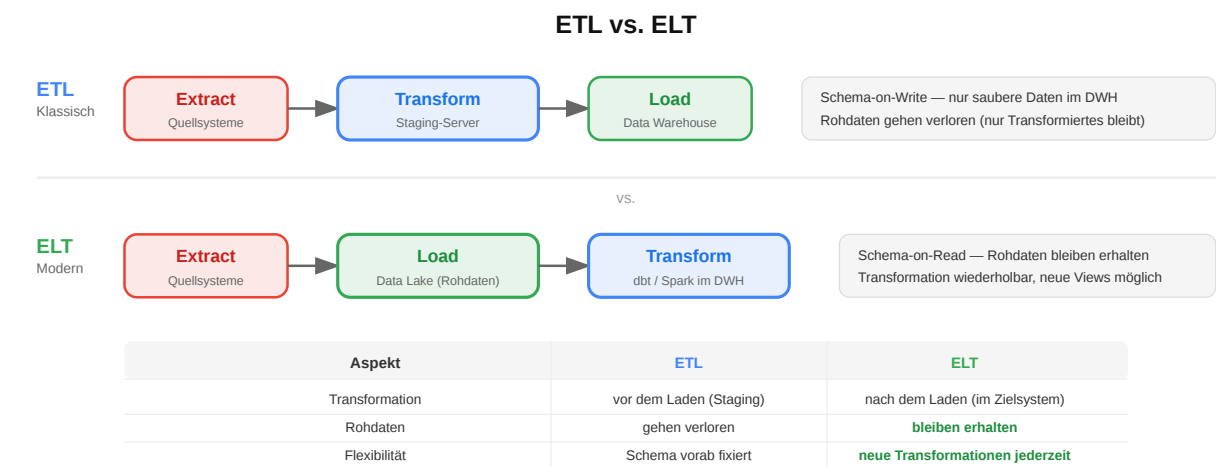
## Batch, Streaming und Orchestrierung

# Leitfrage

Wie verarbeitet man Daten im Petabyte-Maßstab?

# Das klassische Werkzeug: Data Warehouse

Bevor Big Data die Annahmen sprengte, war das **Data Warehouse (DWH)** die Standardantwort für analytische Datenhaltung:



- **ETL:** Daten extrahieren, transformieren, laden — klassisch
- **ELT:** Erst laden (Rohdaten erhalten), dann im Zielsystem transformieren — modern

## OLAP vs. OLTP

| Aspekt   | OLTP          | OLAP            |
|----------|---------------|-----------------|
| Zweck    | Transaktionen | Analysen        |
| Abfragen | viele, kurz   | wenige, komplex |
| Daten    | aktuell       | historisch      |
| Beispiel | Bestellsystem | Quartalsbericht |

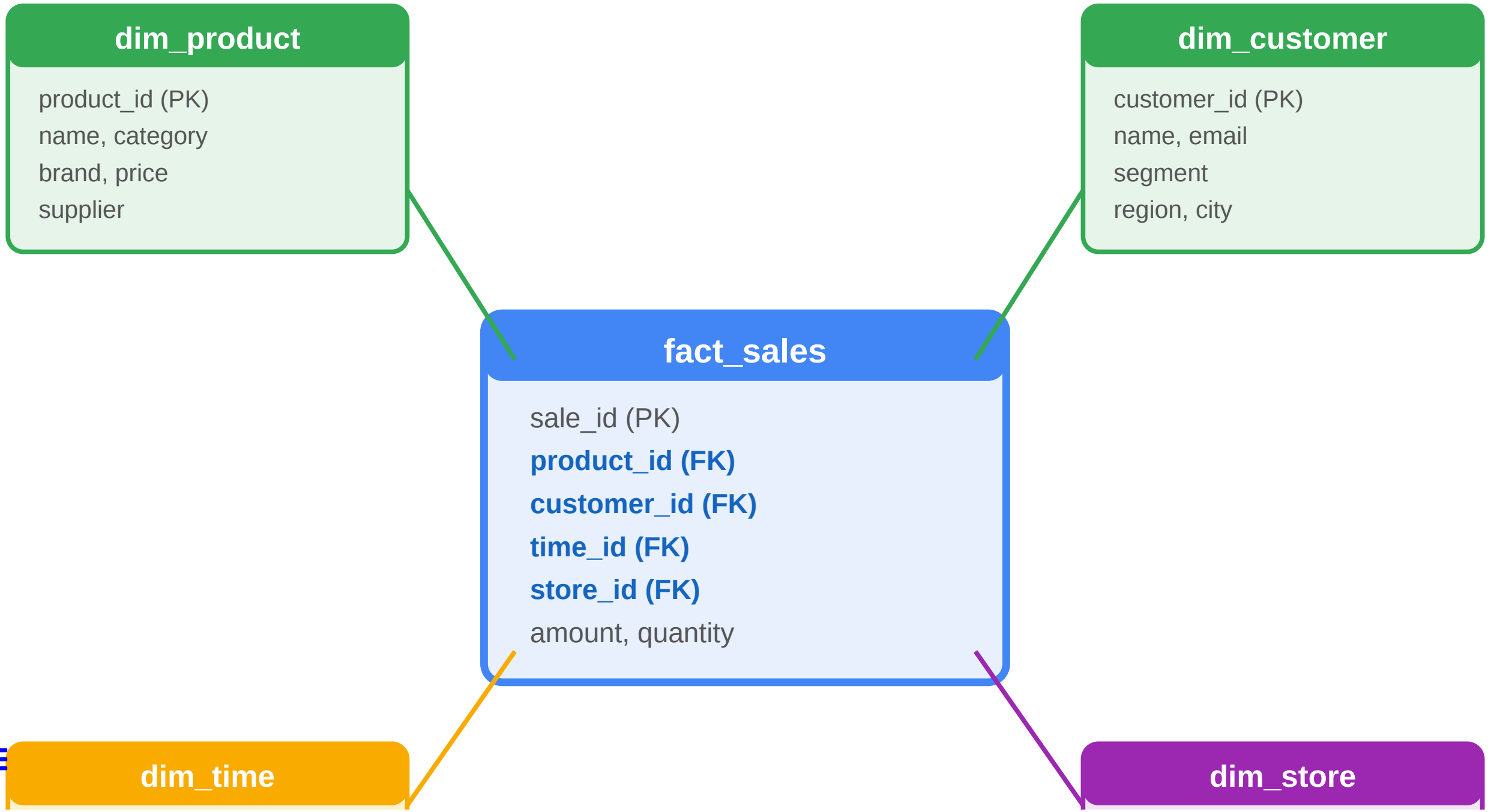
## Star Schema

- **Faktentabelle:** Große Akkumulation numerischer Fakten
- **Dimensionstabellen:** Statische Beschreibungen von Entitäten
- **Maßzahlen:** Aggregate über Dimensionen

# Star Schema: Datenmodell



# Star Schema



# Warum das DWH an Grenzen stößt

Die Fünf Vs stellen die Grundannahmen klassischer DWH-Systeme direkt in Frage:

| Annahme des DWH                            | Herausforderung durch Big Data                           |
|--------------------------------------------|----------------------------------------------------------|
| Daten passen auf einen großen Server       | <b>Volume:</b> Petabyte-Mengen erfordern Verteilung      |
| Schema ist vorab bekannt                   | <b>Variety:</b> JSON, Logs, Graphen, Streams             |
| ETL läuft nachts, Latenz ist akzeptabel    | <b>Velocity:</b> Echtzeit-Anforderungen                  |
| Quelldaten sind verlässlich                | <b>Veracity:</b> Systematische Qualitätsprobleme         |
| Analytischer Mehrwert rechtfertigt Aufwand | <b>Value:</b> Kosten-Nutzen-Relation bei Rohdaten unklar |

# Scaling Up vs. Scaling Out

## Scaling Up vs. Scaling Out

### Vertical Scaling (Scale Up)

2 CPU  
16 GB



**64 CPU**  
2 TB RAM  
10.000 \$/Mo

- + Einfach (ACID funktioniert)
- + Keine verteilte Koordination
- Hardware-Maximum erreicht
- Kosten steigen überproportional

*Typisch: RDBMS, DWH*

### Horizontal Scaling (Scale Out)

Node 1

Node 2

Node 3

Node 4

Node 5

Node 6

... N

je 100 \$/Mo

- + Linear skalierbar
  - + Ausfalltoleranz durch Redundanz
  - Verteilte Koordination nötig
  - CAP-Theorem: C vs. A
- Typisch: HDFS, Spark, Cassandra*

## Vertical Scaling (Scale Up)

- Größere, leistungstärkere Einzelmaschine
- RDBMS mit ACID-Garantien funktionieren gut
- Grenze: Hardware-Maximum irgendwann erreicht
- Kosten steigen überproportional

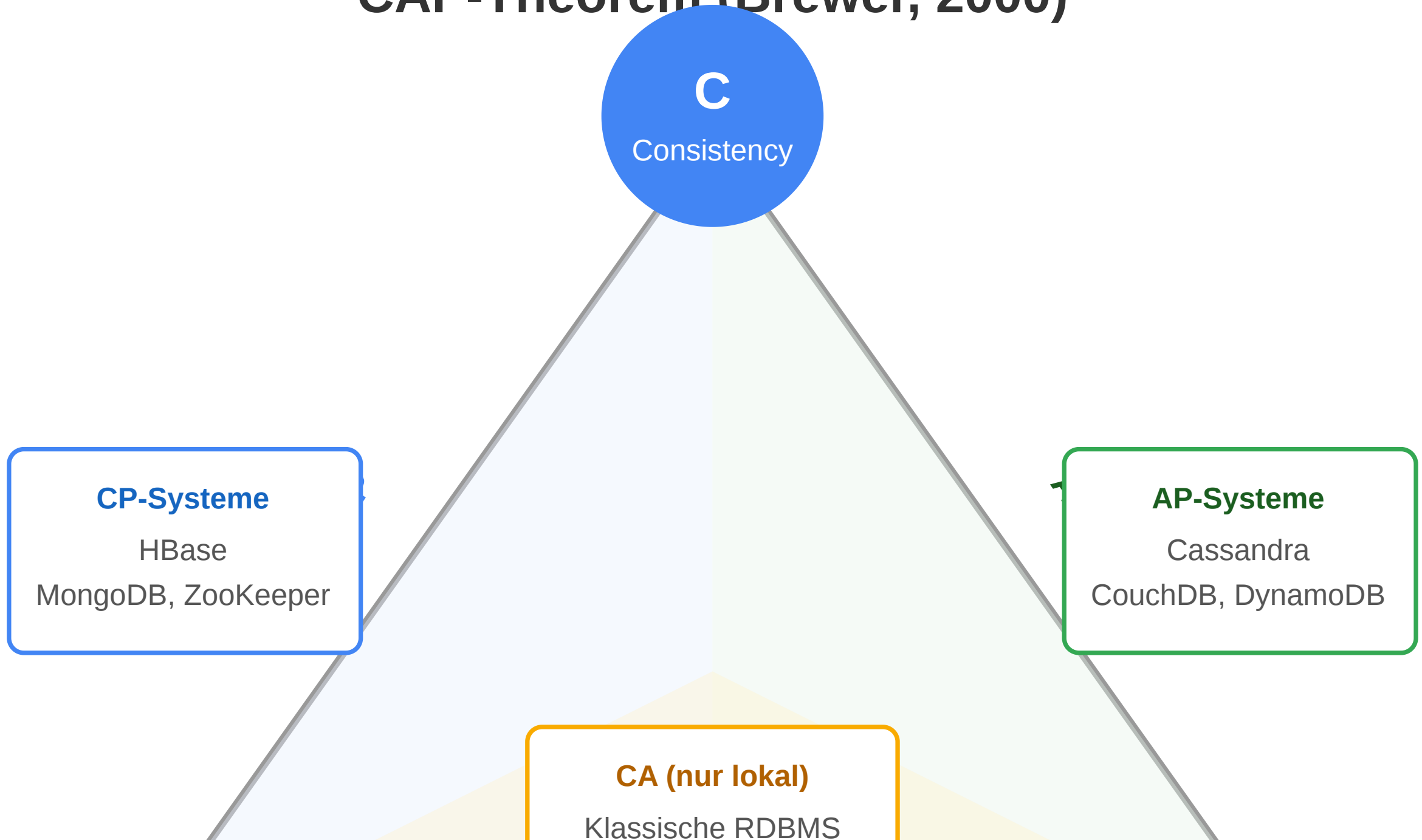
## Horizontal Scaling (Scale Out)

- Viele kleine, günstige Standard-Maschinen
- Jenseits eines Schwellwerts günstiger als Scale Up
- Erfordert verteilte Algorithmen und Koordination
- Ausfalltoleranz durch Redundanz

# CAP-Theorem: Die Grenzen verteilter Systeme



# CAP-Theorem (Brewer, 2000)



**CAP-Theorem (Brewer, 2000):** In einem verteilten System können nicht gleichzeitig alle drei Eigenschaften garantiert werden — es müssen zwei gewählt werden.

CA

A

P

Partition Tol.

| Eigenschaft             | Bedeutung                                                    |
|-------------------------|--------------------------------------------------------------|
| Consistency (C)         | Alle Knoten sehen denselben, aktuellen Zustand               |
| Availability (A)        | Jede Anfrage erhält eine (möglicherweise veraltete) Antwort  |
| Partition Tolerance (P) | System funktioniert auch bei Netzwerkausfall zwischen Knoten |

Systemtypen in der Praxis

| Typ | Systeme                               |
|-----|---------------------------------------|
| CP  | HBase, MongoDB (strong), Zookeeper    |
| AP  | Cassandra, CouchDB, DynamoDB          |
| CA  | Klassische RDBMS (nur ohne Partition) |

lich → Wahl zwischen C und A

In der Praxis:  
P muss immer gegeben sein  
. Die Wahl liegt zwischen  
C  
und  
A  
.

# Der Paradigmenwechsel: SWAPPING zu CLUSTERING

## Traditionell: SWAPPING

Disk → RAM → Cache → Register

- Daten werden Block für Block von Disk geladen
- Einzelne Maschine verarbeitet sequenziell
- Google indiziert ~10 Mrd. Seiten = 200 TB
- Bei 50 MB/s Bandbreite: **40 Tage** nur zum Lesen

Selbst der schnellste Einzelrechner ist zu langsam für Petabyte-Datenmengen.

## Big Data: CLUSTERING

```
[Node 1: 10 GB Chunk] ↔ Computation  
[Node 2: 10 GB Chunk] ↔ Computation  
[Node 3: 10 GB Chunk] ↔ Computation  
... 1000 Nodes
```

- Daten in Chunks aufteilen, auf Nodes verteilen
- 200 TB / 1000 Nodes = **200 GB pro Node**
- Computation kommt zu den Daten

### Data Locality:

Code wird dorthin geschickt, wo die Daten liegen.

# Netzwerk und Ausfalltoleranz im Cluster

## Netzwerk-Bandbreite

- Innerhalb eines Racks: **1 Gbps**
- Rack-übergreifend (Backbone): **2–10 Gbps**
- Daten-Reshuffling ist teuer: 10 TB bei 1 GB/s = **2,8 Stunden**
- Faustregel: **Reshuffling minimieren**

## Ausfalltoleranz

- Bei **1000 Servern**: statistisch ~1 Ausfall pro Tag
- Bei **1.000.000 Maschinen** (Google): ~1000 Ausfälle täglich
- Ausfälle sind keine Ausnahmen — sie sind der **Normalzustand**

### Konsequenz:

- Daten mehrfach replizieren
- Berechnungen bei Ausfall automatisch neu starten
- Kein Job darf an einem einzelnen Knoten hängen

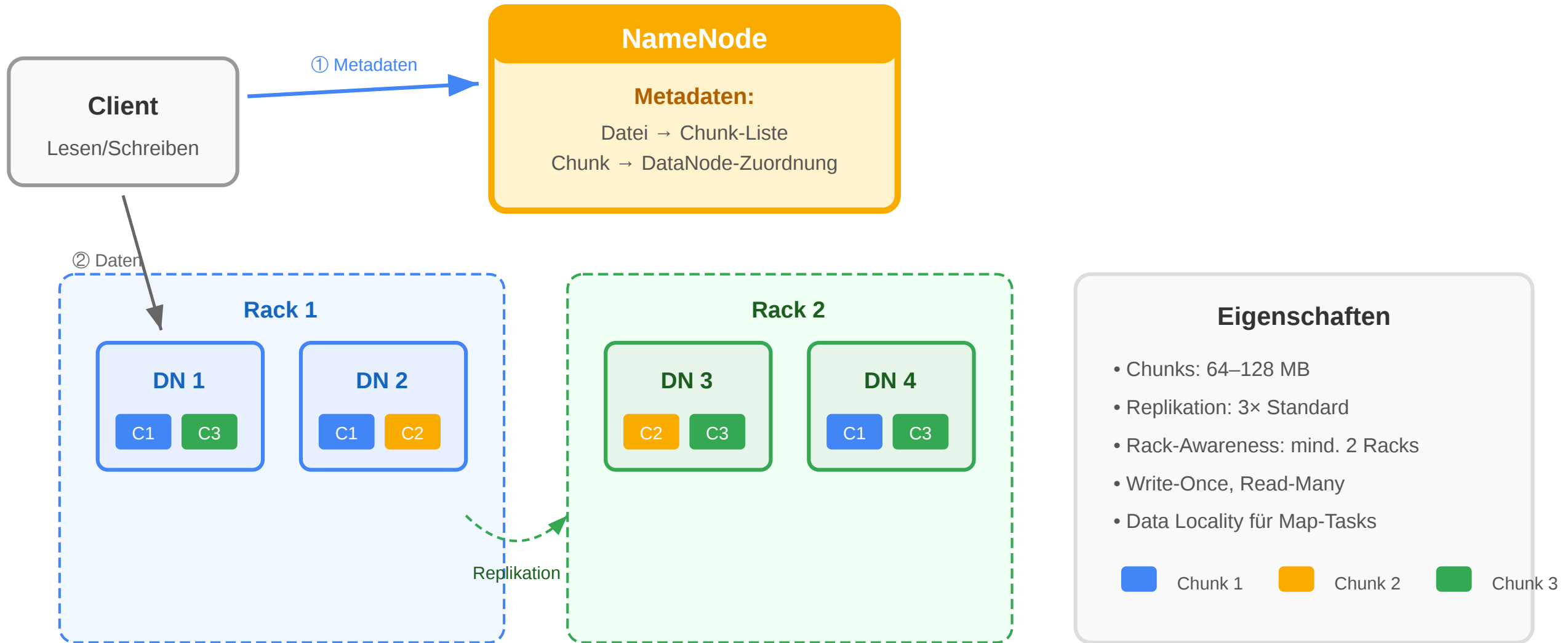
**Design for Failure:** Verteilte Systeme müssen von Anfang an auf Ausfälle ausgelegt sein — nicht nachträglich abgesichert werden.

# HDFS: Verteiltes Dateisystem

Das Hadoop Distributed File System (HDFS) implementiert das Clustering-Paradigma:



# HDFS — Hadoop Distributed File System



# MapReduce: Berechnung auf verteilten Daten

Google's Programmiermodell für verteilte Datenverarbeitung — heute Grundlage von Hadoop:

## Phasen

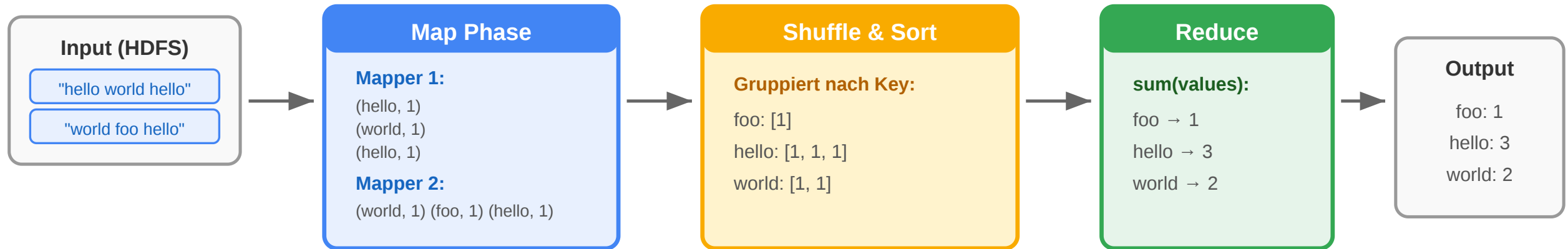
1. **Map:** Für jedes  $\langle \text{key}, \text{value} \rangle$ -Paar: erzeugt eine Menge von  $\langle k', v' \rangle$ -Paaren
2. **Shuffle & Sort:** Intermediäre Paare werden nach Key gruppiert und sortiert
3. **Reduce:** Für jede Gruppe von  $\langle k', v' \rangle$ -Paaren: berechnet ein Aggregat

## Effizienz-Prinzipien

- **Streaming through data:** Sequenzielles Lesen ohne Disk Seeks
- **Data Locality:** Map-Tasks laufen auf dem Knoten mit dem jeweiligen Chunk
- **Automatische Fehlerbehandlung:** Fehlgeschlagene Tasks werden neu gestartet

# MapReduce Beispiel: Wörter zählen

## MapReduce: Word Count Beispiel



### map(key, value):

```
for word in value.split():  
    emit(word, 1)
```

### reduce(key, values):

```
emit(key, sum(values))
```

### Effizienz-Prinzipien:

1. Data Locality — Map läuft auf dem Chunk-Node
2. Streaming — Sequenzielles Lesen, keine Seeks
3. Parallelisierung — alle Mapper gleichzeitig
4. Fehlertoleranz — Tasks automatisch neu starten

```
# Pseudo-Code: MapReduce Wortfrequenz  
def map_fn(key, value):  
    # value = Zeile des Dokuments  
    for word in value.split():  
        emit(word, 1)  
  
def reduce_fn(key, values):  
    # key = Wort, values = [1, 1, 1, ...]  
    emit(key, sum(values))
```

# Stream-Zeitmodelle: Warum Reihenfolge trügt

Im Streaming ist Zeit nicht trivial — dasselbe Event hat drei verschiedene Zeitstempel:

## Stream-Zeitmodelle

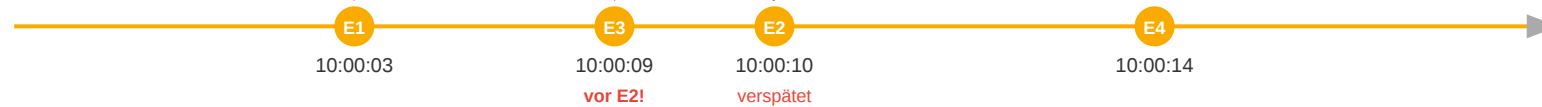
### Event Time

Wann ist das Event tatsächlich passiert?



### Ingestion Time

Wann ist das Event beim Broker (Kafka) eingetroffen?



### Processing Time

Wann hat Flink/Spark das Event verarbeitet?



**Kernproblem: Events treffen nicht in der Reihenfolge ein, in der sie passiert sind. Stream-Systeme brauchen Watermarks und Windowing, um damit umzugehen.**

# Backpressure und Exactly-Once-Semantik

Zwei zentrale Herausforderungen im Stream Processing:

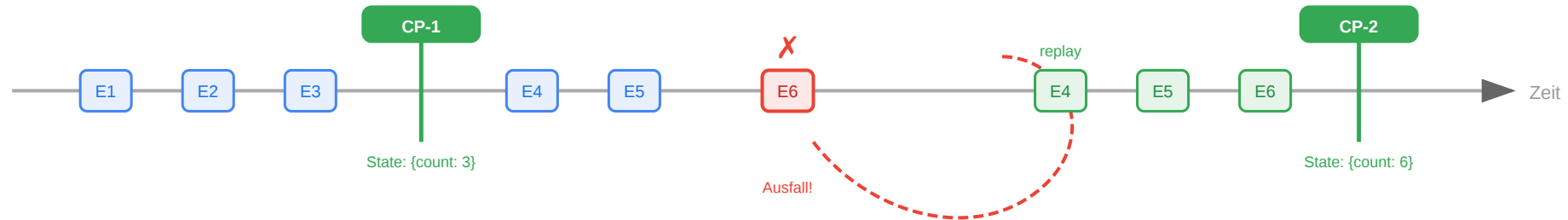
## Backpressure

Was passiert, wenn der Consumer langsamer ist als der Producer?



## Exactly-Once via Checkpointing

Wie garantiert Flink, dass jedes Event genau einmal verarbeitet wird?



**Ergebnis: Jedes Event wird genau einmal gezählt — auch nach einem Ausfall. Checkpoints + Kafka-Replay = Exactly-Once-Semantik.**

# Batch vs. Streaming: Wann was einsetzen?

## Batch-Verarbeitung

- Daten werden gesammelt und periodisch als Block verarbeitet
- **Stärken:** Hoher Durchsatz, Einfachheit, Kosteneffizienz
- **Schwächen:** Hohe Latenz (Minuten bis Stunden)
- **Einsatz:** Reporting, ML-Training, historische Analysen, ETL

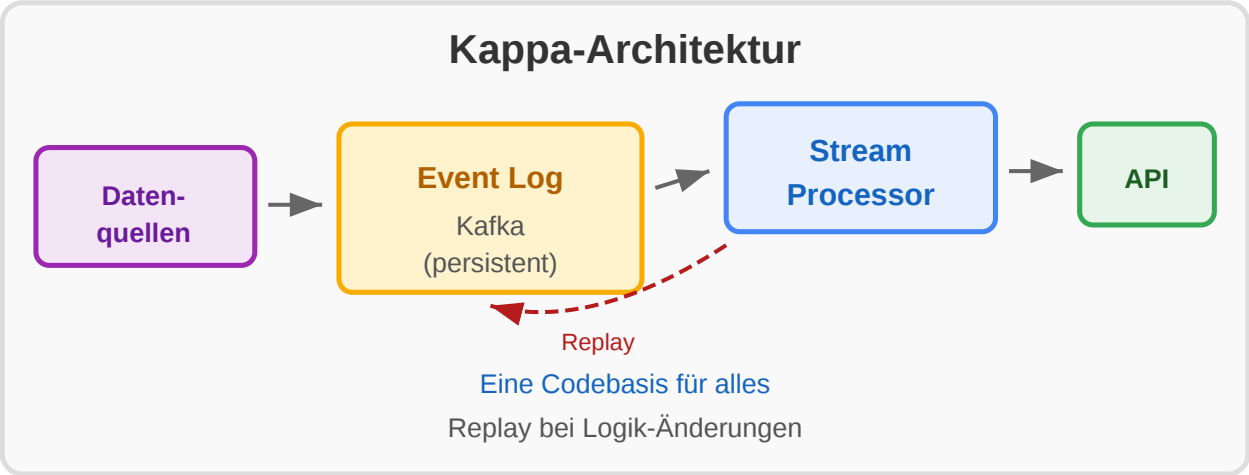
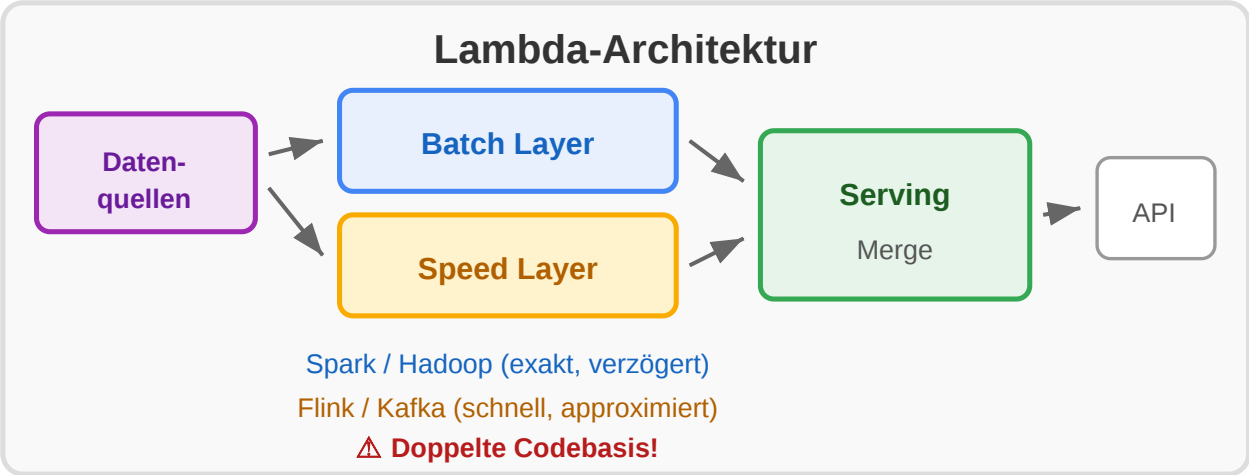
## Stream-Verarbeitung

- Jedes Event wird sofort nach Eintreffen verarbeitet
- **Stärken:** Niedrige Latenz (Millisekunden)
- **Schwächen:** Komplexer (Out-of-Order, Duplikate, Checkpoints)
- **Einsatz:** Betrugserkennung, Alerting, Echtzeit-Empfehlungen

| Kriterium           | Batch                       | Stream                    |
|---------------------|-----------------------------|---------------------------|
| Latenz              | Minuten–Stunden             | Millisekunden–Sekunden    |
| Fehlerbehandlung    | Neuberechnung des Batches   | Checkpoints, Exactly-once |
| Engineering-Aufwand | geringer                    | höher                     |
| Wirtschaftlichkeit  | günstiger bei großen Mengen | teurer bei hohem Volumen  |

# Lambda- und Kappa-Architektur

## Lambda- vs. Kappa-Architektur



| Aspekt                   | Lambda                | Kappa                  |
|--------------------------|-----------------------|------------------------|
| Verarbeitungspfade       | Zwei (Batch + Stream) | Einer (nur Stream)     |
| Codebasis                | Doppelt               | Einfach                |
| Historische Daten        | Batch Layer           | Replay aus Event-Log   |
| Betriebsaufwand          | Hoch                  | Geringer               |
| Genauigkeit (historisch) | Exakt                 | Abhängig von Log-Tiefe |

# Lambda-Architektur: Batch + Stream kombiniert

**Problem:** Wie bekommt man historische Vollständigkeit **und** Echtzeit-Ergebnisse?

## Struktur

- **Batch Layer** (Spark, Hadoop): Vollständige historische Neuberechnung periodisch — exakt, aber verzögert
- **Speed Layer** (Flink, Kafka Streams): Nur neue Events seit letztem Batch — sofortige, approximierte Ergebnisse
- **Serving Layer:** Kombiniert beide Pfade für Abfragen

**Nachteil — Doppelte Codebasis:** Dieselbe Verarbeitungslogik muss für Batch (z.B. Spark) **und** Stream (z.B. Flink) implementiert und synchron gehalten werden. Hoher Betriebsaufwand.

# Kappa-Architektur: Nur Streaming

Idee: Wenn Stream-Systeme robust genug sind — warum noch Batch?

## Struktur

- **Alle Daten** als Events im persistenten Log (Kafka)
- **Eine Codebasis** — dieselbe Logik für neue Events und historische Daten
- **Replay**: Bei Logik-Änderungen gesamtes Log erneut durchlaufen → konsistente Neuberechnung

| Aspekt             | Lambda                | Kappa                |
|--------------------|-----------------------|----------------------|
| Verarbeitungspfade | zwei (Batch + Stream) | einer (nur Stream)   |
| Codebasis          | doppelt               | einfach              |
| Historische Daten  | Batch Layer           | Replay aus Event-Log |
| Betriebsaufwand    | hoch                  | geringer             |

# Mini-Übung: Batch oder Streaming?

Entscheiden Sie für jedes Szenario: **Batch** oder **Streaming** — und begründen Sie kurz.

| Szenario                                                    | Batch oder Streaming? |
|-------------------------------------------------------------|-----------------------|
| Tagesabschluss-Bericht der Verkaufszahlen                   | ?                     |
| Erkennen eines Kreditkartenbetrugs in Echtzeit              | ?                     |
| Wöchentlicher ML-Trainings-Job auf historischen Klick-Daten | ?                     |
| Alerting bei Serverausfall innerhalb von 5 Sekunden         | ?                     |
| Monatliche Kundensegmentierung für E-Mail-Kampagne          | ?                     |

# Musterlösung

| Szenario               | Entscheidung     | Begründung                                     |
|------------------------|------------------|------------------------------------------------|
| Tagesabschluss-Bericht | <b>Batch</b>     | Latenz von Stunden akzeptabel, hoher Durchsatz |
| Kreditkartenbetrug     | <b>Streaming</b> | Muss in Millisekunden entschieden werden       |
| ML-Trainings-Job       | <b>Batch</b>     | Vollständiger historischer Datensatz nötig     |
| Server-Alerting        | <b>Streaming</b> | Reaktionszeit < 5 Sekunden erforderlich        |
| Kundensegmentierung    | <b>Batch</b>     | Monatlicher Rhythmus, kein Echtzeit-Bedarf     |

Batch und Stream sind keine Konkurrenten, sondern komplementäre Werkzeuge. Die Wahl hängt von Latenzbedarf, Durchsatz und Konsistenzanforderungen ab — nicht von technischer Mode.

# Modul 4

---

## Datenqualität und Governance

# Leitfrage

Was macht eine robuste Datenplattform aus — und warum scheitern datengetriebene Systeme oft nicht an Rechenleistung, sondern an Qualität, Verantwortlichkeiten und fehlender Transparenz?

# MapReduce: Effizienz durch Lokalität

Warum ist MapReduce so schnell, obwohl es auf tausenden günstiger Maschinen läuft?

| Prinzip               | Erklärung                                                                                     |
|-----------------------|-----------------------------------------------------------------------------------------------|
| <b>Data Locality</b>  | Map-Task läuft auf dem Node, der den Chunk speichert — kein Netzwerktransfer für Eingabedaten |
| <b>Streaming</b>      | Daten werden sequenziell gelesen, keine Disk-Seekes — maximale Bandbreite                     |
| <b>Pipelining</b>     | Map-Tasks laufen parallel auf allen Nodes — horizontale Skalierung                            |
| <b>Fehlertoleranz</b> | Framework re-startet fehlgeschlagene Tasks automatisch auf Replikat                           |

## Data Locality Principle

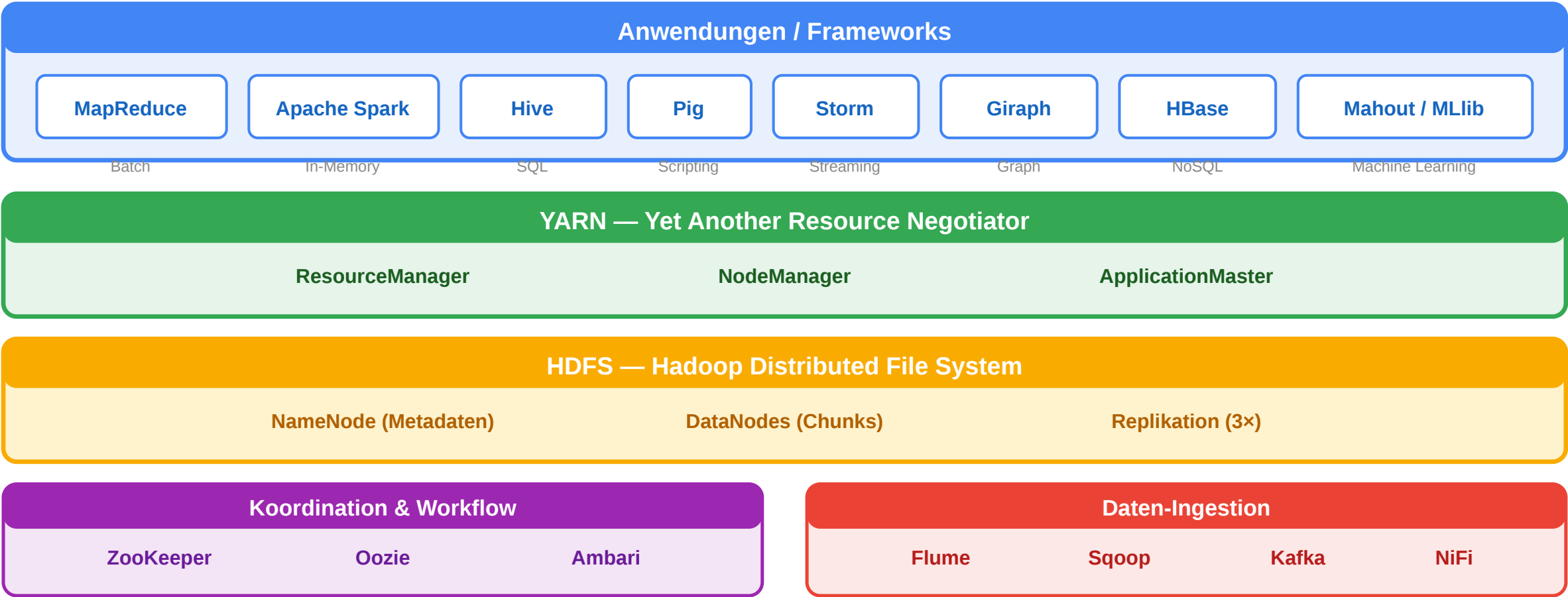
Code ist klein (wenige KB), Daten sind riesig (TB).  
**Bringe die Berechnung zu den Daten**  
, nicht umgekehrt.

### Geeignete Anwendungen:

- Web-Index-Aufbau (Google!)
- Log-Analyse
- ETL-Pipelines
- ML-Feature-Berechnung

# Hadoop Ökosystem

## Hadoop Ökosystem



| Schicht     | Technologien                               | Aufgabe                                          |
|-------------|--------------------------------------------|--------------------------------------------------|
| Anwendungen | MapReduce, Spark, Hive, Pig, Storm, Giraph | Datenverarbeitung (Batch, Streaming, Graph, SQL) |



| Schicht              | Technologien | Aufgabe                                          |
|----------------------|--------------|--------------------------------------------------|
| Ressourcenmanagement | YARN         | Zuweisung von CPU, RAM an Anwendungen im Cluster |
| Speicher             | HDFS         | Verteilte, redundante Datenhaltung in Chunks     |

**Hadoop ≠ MapReduce:** MapReduce ist nur **eine** Anwendung auf Hadoop. Moderne Stacks nutzen meist Spark (In-Memory, 10–100× schneller) auf YARN/HDFS.

# Alternativen zum Scale-Out

## Master/Slave Replikation

- Originaler GFS-Ansatz
- Multi-Master-Replikation möglich
- **INSERT only** — keine Updates/Deletes
- Keine JOINS → reduziert Query-Zeit
- Lesezugriff skaliert durch Replicas

### Einsatz:

Leselastige Systeme, Content-Delivery

## In-Memory-Datenbanken

- Gesamte Datenbank im RAM
- SAP HANA, Oracle In-Memory, Redis
- Extrem geringe Latenz
- Teuer, aber ideal für Echtzeitanalysen

### Einsatz:

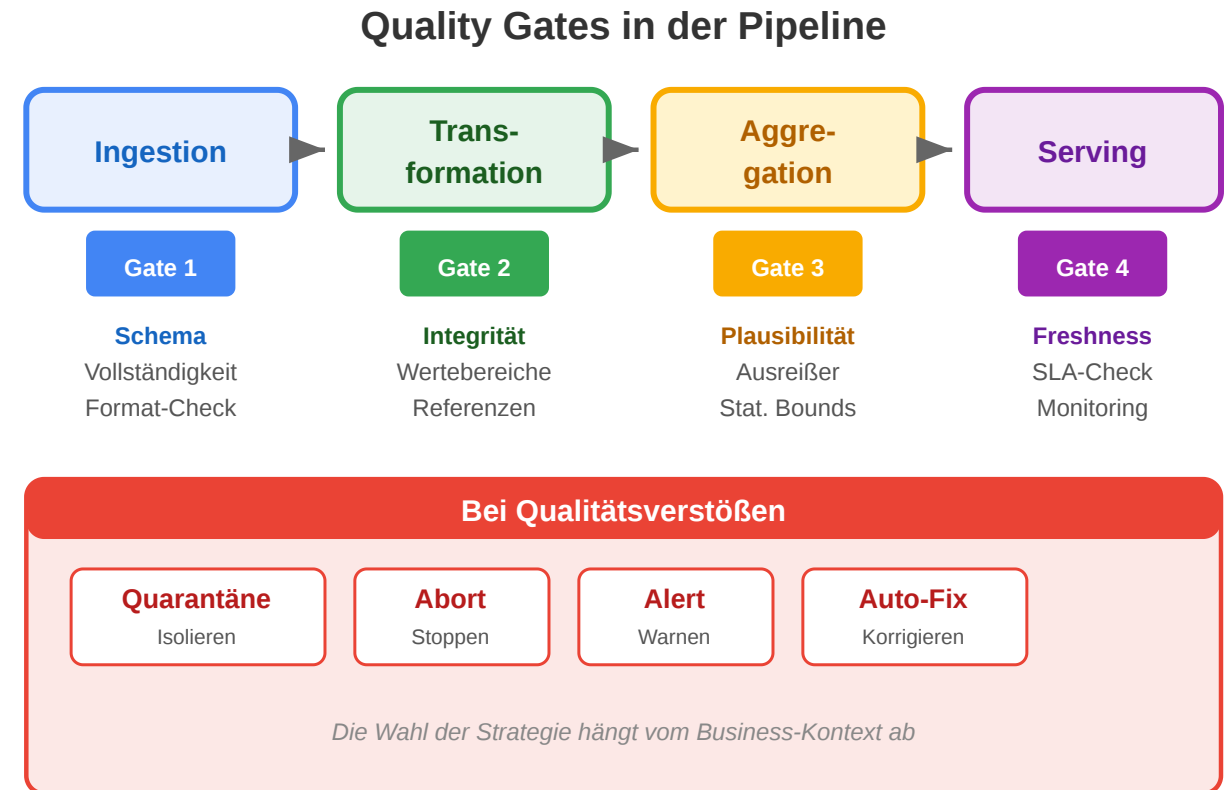
Echtzeit-Dashboards, Trading-Systeme

## Managed Cloud-Services

- BigQuery, Redshift, Snowflake
- Serverless, automatisches Scaling
- Pay-per-Query statt Cluster-Management

# Datenqualität: Die fünf Dimensionen

| Dimension       | Prüffrage                   | Beispiel                            |
|-----------------|-----------------------------|-------------------------------------|
| Vollständigkeit | Fehlen Werte?               | 15 % der Bestellungen ohne PLZ      |
| Konsistenz      | Widersprechen sich Quellen? | CRM sagt 500 Kunden, DWH sagt 480   |
| Aktualität      | Sind Daten frisch genug?    | Lagerbestand 6 Stunden veraltet     |
| Genauigkeit     | Stimmen die Werte?          | GPS-Koordinaten auf falscher Straße |
| Eindeutigkeit   | Gibt es Duplikate?          | Gleicher Kunde mit 3 IDs            |



**Qualität ist kein Feature, sondern ein Prozess:** kontinuierlich messen, automatisiert prüfen, bei Verstößen alarmieren — wie Software-Tests.

# Quality Gates in der Pipeline

| Pipeline-Stufe | Quality Gate                                | Tool-Beispiel                   |
|----------------|---------------------------------------------|---------------------------------|
| Ingestion      | Schema-Validierung, Vollständigkeit, Format | Great Expectations, JSON Schema |
| Transformation | Referentielle Integrität, Wertebereiche     | dbt tests, SQL assertions       |
| Aggregation    | Plausibilitätsprüfung, Ausreißer-Erkennung  | Statistische Bounds, z-Score    |
| Serving        | Freshness-Check, SLA-Monitoring             | Airflow SLAs, Monte Carlo       |

# Was passiert bei Qualitätsverstößen?

| Strategie  | Wann?                                                   |
|------------|---------------------------------------------------------|
| Quarantäne | Fehlerhafte Datensätze in separaten Bereich verschieben |
| Abort      | Pipeline stoppen, wenn kritische Schwelle überschritten |
| Alert      | Team benachrichtigen, Daten weiterverarbeiten           |
| Auto-Fix   | Imputation fehlender Werte, Deduplication               |

**Keine dieser Strategien ist universell richtig** — die Wahl hängt vom Business-Kontext ab: Ein fehlender Sensorwert im IoT-Dashboard ist weniger kritisch als ein fehlerhafter Finanzwert.

# Data Governance: Wer ist verantwortlich?

| Governance-Frage                                   | Warum wichtig?                            |
|----------------------------------------------------|-------------------------------------------|
| <b>Ownership:</b> Wem gehört dieses Datenprodukt?  | Ohne Eigentümer wird kein Bug gefixt      |
| <b>Definitionen:</b> Was bedeutet „aktiver Kunde“? | Verschiedene Teams zählen unterschiedlich |
| <b>Zugriffsrechte:</b> Wer darf was sehen?         | DSGVO, PII, Compliance                    |
| <b>Lineage:</b> Woher kommen diese Daten?          | Debugging, Audit, Vertrauenswürdigkeit    |
| <b>Retention:</b> Wie lange speichern wir?         | Speicherkosten, Recht auf Löschung        |

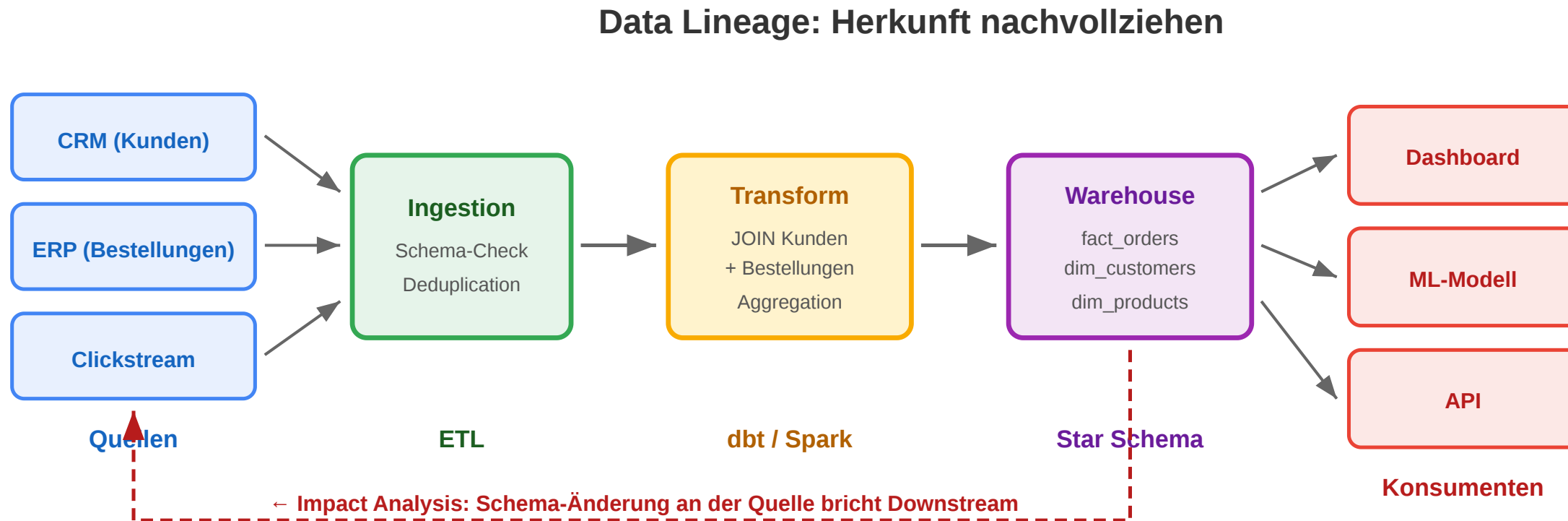
## Data Catalog

Ein  
**Data Catalog**  
ist das „Google für interne Daten“:

- Beschreibung jedes Datensatzes
- Eigentümer und Kontaktperson
- Schema, Datentypen, Beispielwerte
- Lineage-Graph (Herkunft)
- Qualitätsmetriken

**Beispiele:**  
DataHub, Apache Atlas, Alation, Google Data Catalog

# Data Lineage: Herkunft nachvollziehen



## Warum Lineage unverzichtbar ist:

- **Debugging:** Ein Wert im Dashboard ist falsch — welche Transformation hat den Fehler eingeführt?
- **Impact Analysis:** Wir ändern das Schema einer Quelltable — welche Downstream-Systeme brechen?
- **Compliance:** Die DSGVO verlangt Auskunft, wo personenbezogene Daten verarbeitet werden
- **Vertrauen:** Data Scientists müssen wissen, ob sie einem Datensatz vertrauen können

# Mini-Übung: Quality Check Design

Ein Online-Shop hat eine Produktdaten-Pipeline: Lieferanten-CSV → Ingestion → Transformation → Data Warehouse → Produktkatalog-API.

Entwerfen Sie für jede Pipeline-Stufe **einen konkreten Quality Check**:

| Stufe          | Was prüfen Sie? | Erwartetes Ergebnis |
|----------------|-----------------|---------------------|
| Ingestion      | ?               | ?                   |
| Transformation | ?               | ?                   |
| Data Warehouse | ?               | ?                   |
| API            | ?               | ?                   |

# Musterlösung

| Stufe          | Quality Check                                       | Erwartung                                     |
|----------------|-----------------------------------------------------|-----------------------------------------------|
| Ingestion      | CSV hat alle Pflichtfelder (EAN, Name, Preis)       | Keine Zeile ohne EAN                          |
| Transformation | Preis > 0 und < 100.000 €                           | Keine negativen oder absurd hohen Preise      |
| Data Warehouse | Anzahl Produkte ändert sich max. 5 % pro Tag        | Keine Masseninsertions/-löschungen ohne Grund |
| API            | Response enthält $\geq 95$ % der Warehouse-Produkte | Keine stillen Datenverluste beim Serving      |

# Gesamtüberblick: Vom Data Warehouse zu Big Data

| Aspekt       | Klassisches DWH                  | Big Data / Cloud                         |
|--------------|----------------------------------|------------------------------------------|
| Skalierung   | Vertikal (größerer Server)       | Horizontal (mehr Nodes)                  |
| Datenmodell  | Star Schema, strukturiert        | Schema-on-Read, polyglot                 |
| Verarbeitung | ETL → OLAP                       | MapReduce, Spark, Streaming              |
| Speicher     | RDBMS (Oracle, Teradata)         | HDFS, S3, Cloud Storage                  |
| Latenz       | Stunden (Batch-ETL)              | Sekunden (Streaming)                     |
| Kosten       | Hohe Lizenzkosten, wenige Server | Commodity-Hardware, Pay-per-Use          |
| Qualität     | Manuell, selten geprüft          | Automatisierte Quality Gates             |
| Governance   | Zentral (IT-Abteilung)           | Dezentral (Data Mesh / Domain Ownership) |

**Kein Entweder-Oder:** Moderne Datenplattformen kombinieren DWH-Konzepte (Star Schema, SQL) mit Big-Data-Technologien (Spark, Cloud Storage). Das „Lakehouse“-Muster vereint Data Lake und Data Warehouse.



Data Engineering scheitert selten an Rechenleistung. Es scheitert an unklaren Eigentümerschaften, fehlender Lineage, veralteten Definitionen und ungeprüften Annahmen. Qualität, Governance und Transparenz sind keine Add-ons — sie

sind das Fundament jeder belastbaren Datenplattform.

# Wrap-Up

- Data Engineering organisiert den Weg von Datenquellen zu verlässlichen, nutzbaren Datenprodukten.
- Architekturentscheidungen zu Speicher, Verarbeitung und Orchestrierung bestimmen Skalierbarkeit und Reaktionszeit.
- Ohne Datenqualität, Metadaten und Governance werden selbst technisch funktionierende Pipelines fachlich unbrauchbar.